

INVERNADERO INTELIGENTE IoT

Plan de Proyecto — Fase 1

Ponderación:	40 puntos
Período:	2 – 12 junio 2026
Integrantes:	5 personas
Tiempo estimado:	60 horas

Documento generado: junio 2026

1. Resumen Ejecutivo del Proyecto

Campo	Detalle
Curso	ACYE1 – Arquitectura de Computadores y Ensambladores 1
Título	Invernadero Inteligente IoT – Fase 1
Ponderación	40 puntos (40% de la nota final)
Tiempo estimado	60 horas de trabajo

Período de trabajo	2 de junio al 12 de junio de 2026 (11 días)
Integrantes	5 integrantes
Repositorio	ARQUIV1S_G# (auxiliar: PoncheDeFrutas debe ser colaborador)

Descripción General

El proyecto consiste en diseñar e implementar un invernadero inteligente automatizado, utilizando una Raspberry Pi 3 o 4 como unidad principal de control. El sistema integra sensores ambientales y de suelo, actuadores físicos, comunicación IoT mediante MQTT, persistencia de datos en MongoDB Atlas y un dashboard web para monitoreo y control remoto.

El invernadero se construye como una maqueta física funcional dividida en tres zonas: Área de Cultivo 1, Área de Cultivo 2 y un Centro de Control. Adicionalmente, cada integrante del grupo desarrolla un módulo de procesamiento estadístico en lenguaje ensamblador ARM64/AArch64, que se ejecuta sobre datos reales generados por los sensores del invernadero.

Objetivo SMART

Criterio	Descripción
Específico	Diseñar e implementar un sistema IoT de automatización para un invernadero con dos áreas de cultivo y un centro de control.
Medible	Integrando DHT11/22, 2 sensores de suelo, LDR, MQ-2/135, bomba de agua, ventilador, LEDs, buzzer, LCD, botones, MQTT, MongoDB y dashboard web.
Alcanzable	Mediante Raspberry Pi, Python, GPIO, MQTT, MongoDB Atlas y una interfaz web de monitoreo y control.
Realista	Para relacionar sistemas embebidos, automatización agrícola, comunicación IoT, procesamiento ARM64 y visualización de datos en una solución funcional.
A Tiempo	Durante el período del 2 al 12 de junio de 2026 (Fase 1 del proyecto, equivalente a 2 semanas).

2. Listado de Materiales

2.1 Hardware Obligatorio

#	Componente	Cantidad	Función en el sistema
1	Raspberry Pi 3 o 4	1 unidad	Unidad principal de control y procesamiento
2	Sensor DHT11 o DHT22	1 unidad	Medir temperatura y humedad ambiental
3	Sensor de humedad de suelo	2 unidades	Una por cada área de cultivo (seco / normal / saturado)
4	Sensor LDR	1 unidad	Medir nivel de luz ambiental para iluminación automática
5	Sensor MQ-2 o MQ-135	1 unidad	Detectar gas, humo o mala calidad del aire
6	Bomba de agua (mini)	1 unidad	Sistema de riego funcional real
7	Recipiente/depósito de agua	1 unidad	Reservorio de agua para el riego
8	Mangueras / tubería	según diseño	Conducción del agua hacia las áreas de cultivo
9	Ventilador DC o extractor pequeño	1 unidad	Control de temperatura y emergencias por gas
10	LEDs blancos (iluminación)	mínimo 2	Un LED/grupo por cada área de cultivo
11	LED verde	1 unidad	Indicador de estado NORMAL
12	LED amarillo	1 unidad	Indicador de estado ADVERTENCIA o RIEGO_ACTIVADO
13	LED rojo	1 unidad	Indicador de estado EMERGENCIA
14	Buzzer activo	1 unidad	Alarma auditiva para condiciones críticas
15	Pantalla LCD 16x2 o 20x4	1 unidad	Panel local de información y alertas rotativas
16	Pulsadores / botones físicos	mínimo 4	Control local: modo, riego, luces, silenciar buzzer
17	Protoboard(s)	según diseño	Montaje y conexiones de componentes
18	Cables jumper (M-M, M-F, F-F)	varios	Interconexión de componentes
19	Resistencias (220Ω, 10kΩ, etc.)	varias	Protección de LEDs y pull-up/down para botones
20	Transistor NPN o Relé	según diseño	Control de bomba y ventilador desde GPIO

2.2 Hardware Recomendado

#	Componente	Observación
1	Fuente de poder externa (5V / 12V)	Para alimentar bomba o ventilador si consumen más corriente que la Pi
2	Módulo relé de 1 o 2 canales	Alternativa más segura al transistor para controlar bomba/ventilador
3	Convertidor de nivel lógico	Si algún sensor o módulo requiere 5V en lugar de 3.3V
4	MicroSD de alta velocidad (Clase 10+)	Para mejor rendimiento del sistema operativo en la Raspberry Pi

2.3 Software y Herramientas

Herramienta	Tipo	Uso
Python 3	Obligatorio	Lógica de control, GPIO, MQTT, MongoDB, ejecución ARM64
MQTT / MQTTX	Obligatorio	Protocolo de comunicación IoT entre subsistemas
MongoDB Atlas	Obligatorio	Persistencia de lecturas, eventos, comandos y resultados ARM64
Dashboard web (Flask/FastAPI/Node.js)	Obligatorio	Interfaz de monitoreo y control remoto
Ensamblador ARM64 (as/ld/gcc)	Obligatorio	Compilar y ejecutar los 5 módulos estadísticos individuales
GDB	Obligatorio	Depuración individual de cada módulo ARM64
GitHub	Obligatorio	Repositorio ARQUI1V1S_G# con commits individuales por integrante
React / Vue / HTML+JS	Recomendado	Frontend del dashboard web
VS Code	Recomendado	Editor de código para Python, ARM64 y web
Draw.io / Excalidraw	Recomendado	Diagramas de arquitectura, conexiones y flujo de datos
Makefile	Obligatorio	Compilar todos los módulos ARM64 de forma automatizada

2.4 Materiales para la Maqueta Física

La maqueta debe ser una estructura visible de invernadero, NO únicamente componentes en protoboard. Se penaliza con 100% del proyecto si no se presenta la maqueta física.

Material	Descripción / Sugerencia
Estructura del invernadero	Madera balsa, cartón grueso, PVC, acrílico o materiales reciclados. Debe tener forma de invernadero.
División de zonas	Separadores físicos visibles para Área 1, Área 2 y Centro de Control.
Plantas	Plantas reales pequeñas o simuladas con suelo real o material para maceta.
Recipiente de cultivo	Maceta, bandeja o caja con sustrato/tierra para cada área.
Sistema de riego	Mangueras, tuberías de plástico delgado o jeringas para conducir el agua de la bomba.

Etiquetas / rotulación

Identificar claramente cada zona y sensor de la maqueta.

3. Plan de Sprints (2 al 12 de junio de 2026)

Período total: 11 días de trabajo. La evaluación y defensa técnica es el Día 13. Se plantean 4 sprints que cubren todas las actividades del proyecto, incluyendo maqueta, código, ARM64, documentación y video.

Sprint 1 — Diseño, Maqueta y Sensores Básicos | Días 1-3 (2–4 junio)

#	Tarea	Responsable(s)	Entregable
1	Definir arquitectura del sistema: zonas, sensores, actuadores, topics MQTT y colecciones MongoDB	Todos	Diagrama de arquitectura
2	Asignar módulos ARM64 individuales (Integrantes 1–5)	Todos	Asignación documentada
3	Crear repositorio GitHub con estructura de carpetas (python/, arm64/, dashboard/, docs/)	Integrante líder	Repositorio listo + auxiliar como colaborador
4	Construir estructura física del invernadero (3 zonas visibles)	Todos	Maqueta estructural lista
5	Instalar y configurar Raspberry Pi (SO, Python, librerías GPIO, MQTT, pymongo)	1-2 integrantes	Pi configurada y funcional
6	Prueba individual: sensores DHT11/22, LDR, humedad de suelo, MQ-2/135	2-3 integrantes	Scripts de prueba individuales
7	Instalar sensores en la maqueta y verificar lecturas desde Python	Todos	Sensores físicamente instalados
8	Iniciar diseño del dashboard (wireframes / mockups)	1 integrante	Mockups del dashboard

Sprint 2 — Lógica Python, Actuadores, MQTT y MongoDB | Días 4-7 (5–8 junio)

#	Tarea	Responsable(s)	Entregable
1	Implementar bucle principal Python: lectura de sensores, estados globales (NORMAL, ADVERTENCIA, EMERGENCIA, RIEGO_ACTIVO, MODO_MANUAL)	2 integrantes	main.py funcional
2	Implementar control de actuadores: bomba de agua (riego por área), ventilador, LEDs y buzzer	2 integrantes	Scripts de actuadores
3	Integrar riego real con bomba de agua, mangueras y recipiente. Prueba de riego automático y manual	2 integrantes	Riego físico funcionando
4	Implementar pantalla LCD: información rotativa y alertas críticas fijas	1 integrante	LCD mostrando datos
5	Implementar botones físicos: modo auto/manual, riego manual, luces, silenciar buzzer	1 integrante	Botones funcionales
6	Configurar broker MQTT y definir todos los topics (sensores, actuadores, estado, control)	1-2 integrantes	MQTT publicando y recibiendo
7	Configurar MongoDB Atlas: crear colecciones (sensor_readings, events, commands, system_status, actuador_logs)	1 integrante	BD con colecciones creadas

8	Integrar publicación MQTT + almacenamiento MongoDB en el bucle principal de Python	2 integrantes	Datos fluyendo a MQTT y MongoDB
9	Probar flujo completo: sensor → Python → MQTT → MongoDB	Todos	Evidencia de flujo IoT
10	Iniciar utils.s (biblioteca ARM64 común: lectura CSV, conversión ASCII, manejo de buffers)	Todos (juntos)	utils.s base funcional

Sprint 3 — Dashboard, ARM64 Individual y lecturas.csv | Días 8-10 (9–11 junio)

#	Tarea	Responsable(s)	Entregable
1	Desarrollar dashboard web: panel principal (lecturas en tiempo real, estado global)	1-2 integrantes	Panel principal del dashboard
2	Implementar gráficas históricas en el dashboard (temperatura, humedad, suelo, luz, gas)	1 integrante	Gráficas funcionando
3	Implementar controles remotos en dashboard (riego, luces, ventilador, alarma, modo auto/manual)	1-2 integrantes	Controles remotos funcionales
4	Implementar historial de eventos y comandos en el dashboard	1 integrante	Sección historial
5	Generar lecturas.csv con exactamente 30 lecturas reales del sistema (Python)	1 integrante	lecturas.csv generado
6	Finalizar utils.s: lectura completa de lecturas.csv, parsing de columnas, escritura de resultados	Todos (juntos)	utils.s completo y funcional
7	Integrante 1: desarrollar modulo_1_media.s (media aritmética ponderada)	Integrante 1	modulo_1_media.s + resultado_media.txt
8	Integrante 2: desarrollar modulo_2_varianza.s (varianza y desviación estándar)	Integrante 2	modulo_2_varianza.s + resultado_varianza.txt
9	Integrante 3: desarrollar modulo_3_anomalias.s (detección de anomalías z-score)	Integrante 3	modulo_3_anomalias.s + resultado_anomalias.txt
10	Integrante 4: desarrollar modulo_4_prediccion.s (predicción lineal simple)	Integrante 4	modulo_4_prediccion.s + resultado_prediccion.txt
11	Integrante 5: desarrollar modulo_5_tendencia.s (tendencia acumulada avanzada)	Integrante 5	modulo_5_tendencia.s + resultado_tendencia.txt
12	Compilar y ejecutar cada módulo ARM64 de forma independiente. Verificar archivo de salida .txt	Cada integrante	5 módulos compilando y ejecutando
13	Crear Makefile para compilar todos los módulos ARM64	1 integrante	Makefile funcional

Sprint 4 — Integración ARM64, Pruebas Finales, Documentación y Video | Días 11-12 (11–12 junio)

#	Tarea	Responsable(s)	Entregable
1	Script Python que ejecuta los 5 módulos ARM64 y lee sus archivos de salida .txt	1-2 integrantes	Script de integración ARM64
2	Almacenar resultados ARM64 en colección arm64_results de MongoDB	1 integrante	Colección arm64_results poblada
3	Agregar sección 'Análisis ARM64' al dashboard (mostrar resultados de los 5 módulos)	1 integrante	Sección ARM64 en dashboard
4	Prueba de flujo completo: sensores → Python → lecturas.csv → ARM64 → resultados → MongoDB → dashboard	Todos	Flujo end-to-end verificado
5	Depuración GDB: cada integrante realiza y documenta su sesión de depuración de su módulo ARM64	Cada integrante	5 evidencias de GDB (capturas de pantalla)
6	Prueba de integración general del sistema completo (hardware + software)	Todos	Sistema integrado estable
7	Grabar video demostrativo (sensores, actuadores, riego, LCD, botones, MQTT, MongoDB, dashboard, ARM64)	Todos	Video demostrativo completo
8	Redactar documentación técnica (subsistemas, topics, colecciones, diagramas, decisiones de diseño)	2 integrantes	Informe técnico final
9	Documentación individual ARM64 (algoritmo, registros, ciclos, saltos, subrutinas, entrada/salida)	Cada integrante	5 documentaciones individuales
10	Diagramas finales (arquitectura general, conexiones físicas, flujo de datos, flujo MQTT, modelo MongoDB)	1-2 integrantes	Conjunto de diagramas
11	Revisión final del repositorio: estructura limpia, commits por integrante verificables, README completo	Todos	Repositorio entregado

4. Distribución de Trabajo (5 Integrantes)

La distribución está diseñada para que cada integrante tenga responsabilidades equilibradas (~12 horas de trabajo cada uno), cubra tanto hardware/software como el módulo ARM64 individual obligatorio, y pueda defender técnicamente su parte en la evaluación.

Integrante	Área principal de responsabilidad	Tareas específicas	Módulo ARM64
Integrante 1	Arquitectura del sistema + Python núcleo	Diseño de arquitectura y diagramas. Bucle principal Python: lectura de sensores, máquina de estados global, publicación MQTT, almacenamiento MongoDB.	modulo_1_media.s — Media aritmética ponderada
Integrante 2	Maqueta física + Sistema de riego	Construcción y organización de la maqueta (3 zonas). Implementación del sistema de riego real (bomba, mangueras, recipiente, control por área, protecciones).	modulo_2_varianza.s — Varianza y desviación estándar
Integrante 3	Centro de control físico + Dashboard web	LCD con información rotativa y alertas. Botones físicos (4 obligatorios). LEDs de estado. Desarrollo completo del dashboard web (panel, gráficas, controles remotos, historial, sección ARM64).	modulo_3_anomalias.s — Detección de anomalías (z-score)
Integrante 4	Sensores ambientales + Actuadores ventilación/alarma	Integración y prueba de todos los sensores (DHT, LDR, MQ-2, suelo). Control del ventilador e iluminación automática. Sistema de alarma (buzzer + LED rojo). Generación de lecturas.csv con 30 datos reales.	modulo_4_prediccion.s — Predicción lineal simple
Integrante 5	Integración ARM64 + Documentación técnica	utils.s (biblioteca común ARM64). Script Python de integración (ejecutar módulos, leer salidas, guardar en MongoDB). Makefile. Informe técnico final, diagramas y video demostrativo.	modulo_5_tendencia.s — Tendencia acumulada avanzada

Tareas Compartidas (Responsabilidad de Todo el Equipo)

Tarea compartida	Cuándo
Revisión y prueba cruzada de cada subsistema integrado	Sprint 2 y 3
Desarrollo colaborativo inicial de utils.s (biblioteca común ARM64)	Sprint 2 (Días 6-8)
Pruebas de integración completa del sistema end-to-end	Sprint 4
Grabación del video demostrativo	Día 12
Revisión final del repositorio y commits	Día 12

Notas Importantes sobre la Distribución

- **Cada integrante DEBE poder explicar y defender su módulo ARM64 individualmente. No se acepta que otro lo defienda.**
- Cada integrante debe tener commits individuales verificables en el repositorio GitHub.
- El sistema de riego debe ser REAL y funcional (no simulado con LEDs). Es uno de los requisitos más críticos de la rúbrica.
- La maqueta física es obligatoria. Sin ella el grupo obtiene 100% de penalización.
- El auxiliar (PoncheDeFrutas en GitHub) debe ser colaborador del repositorio desde el inicio.
- Python NO puede realizar los cálculos estadísticos que corresponden a los módulos ARM64. De lo contrario se obtiene 0 en ARM64 e integración.
- Todos los módulos ARM64 deben enlazarse con utils.s y procesar exactamente 30 datos de lecturas.csv.

5. Checklist de Entregables Obligatorios

✓	Entregable / Requisito	Responsable	Sprint
[]	Maqueta física de invernadero (2 áreas de cultivo + centro de control)	Todos	S1
[]	Repositorio GitHub creado con estructura correcta y auxiliar como colaborador	I1	S1
[]	Raspberry Pi configurada y funcionando con todos los sensores	I4	S1-2
[]	Riego real funcional con bomba de agua integrada a la maqueta	I2	S2
[]	Ventilador, LEDs de estado, buzzer e iluminación funcionando	I4	S2
[]	LCD con información rotativa y alertas críticas	I3	S2
[]	4 botones físicos funcionales	I3	S2
[]	Python GPIO leyendo sensores y controlando actuadores	I1	S2
[]	MQTT publicando lecturas y recibiendo comandos (todos los topics)	I1	S2
[]	MongoDB Atlas con todas las colecciones y documentos con timestamp	I1	S2
[]	Dashboard web con panel, gráficas, controles remotos, historial y sección ARM64	I3	S3
[]	lecturas.csv generado con exactamente 30 lecturas reales	I4	S3
[]	utils.s funcional (lectura CSV, parsing, conversión ASCII, escritura .txt)	I5	S2-3
[]	modulo_1_media.s compilando y generando resultado_media.txt	I1	S3
[]	modulo_2_varianza.s compilando y generando resultado_varianza.txt	I2	S3
[]	modulo_3_anomalias.s compilando y generando resultado_anomalias.txt	I3	S3
[]	modulo_4_prediccion.s compilando y generando resultado_prediccion.txt	I4	S3
[]	modulo_5_tendencia.s compilando y generando resultado_tendencia.txt	I5	S3
[]	Makefile que compila todos los módulos ARM64	I5	S3
[]	Script Python de integración ARM64 (ejecuta módulos, lee .txt, guarda en MongoDB)	I5	S4
[]	Colección arm64_results en MongoDB con resultados de los 5 módulos	I5	S4
[]	Evidencia de depuración GDB por cada integrante (5 evidencias)	Cada uno	S4
[]	Video demostrativo completo del sistema	Todos	S4
[]	Documentación técnica (subsistemas, topics, colecciones, diagramas)	I5	S4

[]	Documentación individual ARM64 por cada integrante (5 documentos)	Cada uno	S4
[]	Commits individuales verificables por cada integrante en el repositorio	Todos	Continuo